DATA-260 Homework 8 — Kafka Request/Response, Agent Pipeline, Failure Behavior

Xu Wang_018319275 · SJSU · DATA 260

GitHub repository: https://github.com/xuwang-goldenwater/data260-9275

Section 0 — Personal Configuration

Value	Definition	My value
SID4	Last four digits of SJSU ID	9275
PORT_BASE	8000 + (SID4 mod 900)	8275
PREFIX	"s" + SID4	s9275
SEED	SID4	9275
VERIFY_SEED	260000 + SID4	269275
DOMAIN_ID	SID4 mod 8	3 — Grocery supply and recall notices (entity: RecallNotice)

Item	Value
Hardware	MacBook Pro, Apple M1, 8 GB RAM, macOS; Docker Desktop
Kafka	apache/kafka:3.8.0, 1 broker, KRaft mode (docker-compose.yml)
Local model	qwen2.5:3b via Ollama, temperature 0, seed 9275 (Part 2 agents)
Kafka client	confluent-kafka (Python)
Code commit verified	cdf9602 — make verify-hw08: 30 passed / 0 failed
Tag	hw8

Part 1 — RecallNotice Request/Response over Kafka

The domain logic runs only in the worker. The client (homework_demo.py) and the FastAPI wrapper never touch the data; they send a request message and wait for the response with the same correlation_id. Operations: CREATE_RECALL_NOTICE and GET_RECALL_NOTICE; the id field is recall_notice_id. Entities are kept in a plain Python dict inside the worker.

Topics and message schemas

Topic	Written by	Read by
s9275_recall_notice_requests	homework_demo.py / FastAPI	worker (group s9275_recall_worker)
s9275_recall_notice_responses	worker (topic named in reply_to)	homework_demo.py / FastAPI

Request schema (CREATE and GET)

```
{"correlation_id": "94cf8dce-...", "reply_to": "s9275_recall_notice_responses",
 "operation": "CREATE_RECALL_NOTICE",
 "data": {"productName": "...", "recallingFirm": "...", "submitterEmail": "...",
          "description": "...", "category": "Undeclared Allergen", "agreedToTerms": true}}

{"correlation_id": "...", "reply_to": "s9275_recall_notice_responses",
 "operation": "GET_RECALL_NOTICE", "data": {"recall_notice_id": "08b73dc8-..."}}
```

Response schema (success and error)

```
{"correlation_id": "94cf8dce-...", "success": true,
 "recall_notice_id": "08b73dc8-...", "message": "Recall notice created"}
{"correlation_id": "6bbe28f4-...", "success": true, "recall_notice": {"recall_notice_id":
"08b73dc8-...", ...}}
{"correlation_id": "err-notfound-1", "success": false, "error": "Recall notice not found"}
```

correlation_id flow

The client creates the correlation_id once, before sending (one per request: CREATE and GET each get their own). The FastAPI wrapper forwards a caller's X-Correlation-ID header unchanged and only creates one when it is missing. The worker copies it into every response, error responses included. The client and the API match a response to its request only by this value.

Code — worker (validate, process, reply to reply_to, commit after the reply)

code/kafka_app/recall_service.py — handle_request() (validation in validate_notice())

```
    if operation == "CREATE_RECALL_NOTICE":
        problem = validate_notice(data)
        if problem:
            return error(problem)
        notice_id = str(uuid.uuid4())
        ...
    if operation == "GET_RECALL_NOTICE":
        notice_id = data.get("recall_notice_id")
        if not isinstance(notice_id, str) or not notice_id.strip():
            return error("Missing field: recall_notice_id")
        record = store.get(notice_id.strip())
        if record is None:
            return error("Recall notice not found")
        return {"correlation_id": cid, "success": True, "recall_notice": record}

    return error("Unknown operation")
```

code/kafka_app/worker.py — main loop

```
        response = handle_request(request, STORE)
        log("PROCESSED", f"cid={response['correlation_id']} op={request.get('operation')} "
                         f"success={response['success']} store_size={len(STORE)}")

        reply_to = request.get("reply_to") or RESPONSE_TOPIC
        producer.produce(reply_to, key=str(response["correlation_id"]),
                         value=json.dumps(response))
        producer.flush(10)
        log("SENT", f"topic={reply_to} {json.dumps(response)}")

        consumer.commit(message=msg, asynchronous=False)
```

Code — FastAPI wrapper (waits for the matching correlation_id)

code/kafka_app/api.py

```
async def call_worker(operation: str, data: Dict[str, Any], cid: Optional[str]) -> Dict[str, Any]:
    """Send one request to Kafka and wait for its response."""
    cid = cid or str(uuid.uuid4())
    request = {"correlation_id": cid, "reply_to": RESPONSE_TOPIC,
               "operation": operation, "data": data}
    fut = LOOP.create_future()
    PENDING[cid] = fut
    try:
        log("SENT", f"topic={REQUEST_TOPIC} {json.dumps(request)}")
        producer.produce(REQUEST_TOPIC, key=cid, value=json.dumps(request))
        await run_in_threadpool(producer.flush, 5)
        response = await asyncio.wait_for(fut, TIMEOUT_S)
    except asyncio.TimeoutError:
        log("TIMEOUT", f"cid={cid} no response after {TIMEOUT_S}s")
        raise HTTPException(504, detail={"correlation_id": cid,
                                         "error": "No response from worker"})
    finally:
        PENDING.pop(cid, None)
    log("RETURNED", f"cid={cid} success={response.get('success')} -> HTTP caller")
    return response
```

V-1 Docker containers

docker compose runs kafka, a one-shot kafka-init job that creates all seven topics, the worker and the API.

```
Xu Wang - SID4 9275
CONTAINER ID   IMAGE              COMMAND                  CREATED        STATUS                  PORTS
                          NAMES
5309ec14f8c3   s9275-kafka-app    "python api.py"          3 hours ago    Up 3 hours              127.0.0.1:8275
->8275/tcp                s9275-api
702bc25cdc70   s9275-kafka-app    "python worker.py"       3 hours ago    Up 3 hours
                          s9275-worker
59d862d02c4f   apache/kafka:3.8.0   "/__cacert_entrypoin…"   3 hours ago    Up 3 hours (healthy)   0.0.0.0:9092->
9092/tcp, [::]:9092->9092/tcp   s9275-kafka
360a8355521e   postgres:13        "docker-entrypoint.s…"   6 months ago   Up 4 hours (healthy)   5432/tcp
                          lab2-postgres-1
16:11:22.535 [worker] START    bootstrap=kafka:29092 topic=s9275_recall_notice_requests group=s9275_recall_worke
r
16:11:22.535 [worker] READY    Consumer is ready and listening...
16:11:38.883 [worker] RECEIVED  partition=0 offset=0 {"correlation_id": "94cf8dce-408f-4be0-aece-11c784362f26", "
reply_to": "s9275_recall_notice_responses", "operation": "CREATE_RECALL_NOTICE", "data": {"productName": "Sunrise
 Valley Creamy Peanut Butter, 16 oz jar", "recallingFirm": "Sunrise Valley Foods, Inc.", "submitterEmail": "quali
ty@sunrisevalleyfoods.com", "category": "Undeclared Allergen", "description": "Lots SV2451-SV2458, best-by 2027-0
3-11 to 2027-03-18, may contain undeclared milk from a shared production line. Do not eat; return to the store fo
r a full refund.", "agreedToTerms": true}}
```

V-2 / V-3 / V-4 CREATE -> GET, message flow, full sequence (homework_demo.py)

homework_demo.py logs its own SENT and RECEIVED lines and pulls the worker's lines for the same correlation_id from docker logs, so the complete sequence prints in order: client SENT -> worker RECEIVED -> worker PROCESSED -> worker SENT -> client RECEIVED -> correlation_id check.

```
=== HW8 Part 1 demo - Xu Wang (SID4 9275, PREFIX s9275) ===

--- CREATE_RECALL_NOTICE ---
18:44:45.295 [client] CID       created correlation_id=4a8926ad-d40c-4cc5-98b6-c401920c5aa9
18:44:45.295 [client] SENT      topic=s9275_recall_notice_requests {"correlation_id": "4a8926ad-d40c-4cc5-98b6-c401920c5aa9", "reply_to": "s9275_recall_notice_res
ponses", "operation": "CREATE_RECALL_NOTICE", "data": {"productName": "Sunrise Valley Creamy Peanut Butter, 16 oz jar", "recallingFirm": "Sunrise Valley Foods, In
c.", "submitterEmail": "quality@sunrisevalleyfoods.com", "category": "Undeclared Allergen", "description": "Lots SV2451-SV2458, best-by 2027-03-11 to 2027-03-18,
may contain undeclared milk from a shared production line. Do not eat; return to the store for a full refund.", "agreedToTerms": true}}
18:44:45.317 [worker] RECEIVED  partition=0 offset=20 {"correlation_id": "4a8926ad-d40c-4cc5-98b6-c401920c5aa9", "reply_to": "s9275_recall_notice_responses", "ope
ration": "CREATE_RECALL_NOTICE", "data": {"productName": "Sunrise Valley Creamy Peanut Butter, 16 oz jar", "recallingFirm": "Sunrise Valley Foods, Inc.", "submitt
erEmail": "quality@sunrisevalleyfoods.com", "category": "Undeclared Allergen", "description": "Lots SV2451-SV2458, best-by 2027-03-11 to 2027-03-18, may contain u
ndeclared milk from a shared production line. Do not eat; return to the store for a full refund.", "agreedToTerms": true}}
18:44:45.326 [worker] PROCESSED cid=4a8926ad-d40c-4cc5-98b6-c401920c5aa9 op=CREATE_RECALL_NOTICE success=True store_size=6
18:44:45.330 [worker] SENT      topic=s9275_recall_notice_responses {"correlation_id": "4a8926ad-d40c-4cc5-98b6-c401920c5aa9", "success": true, "recall_notice_id"
: "aa68813e-e4fc-4bcb-99a6-a3331da049ba", "message": "Recall notice created"}
18:44:45.381 [client] RECEIVED  topic=s9275_recall_notice_responses {"correlation_id": "4a8926ad-d40c-4cc5-98b6-c401920c5aa9", "success": true, "recall_notice_id"
: "aa68813e-e4fc-4bcb-99a6-a3331da049ba", "message": "Recall notice created"}
18:44:45.381 [client] CHECK     correlation_id preserved: True
18:44:45.381 [client] EXTRACT   recall_notice_id=aa68813e-e4fc-4bcb-99a6-a3331da049ba

--- GET_RECALL_NOTICE ---
18:44:45.381 [client] CID       created correlation_id=d33da180-635b-4d46-ac9c-a94397dcccdd
18:44:45.381 [client] SENT      topic=s9275_recall_notice_requests {"correlation_id": "d33da180-635b-4d46-ac9c-a94397dcccdd", "reply_to": "s9275_recall_notice_res
ponses", "operation": "GET_RECALL_NOTICE", "data": {"recall_notice_id": "aa68813e-e4fc-4bcb-99a6-a3331da049ba"}}
18:44:45.382 [worker] RECEIVED  partition=0 offset=21 {"correlation_id": "d33da180-635b-4d46-ac9c-a94397dcccdd", "reply_to": "s9275_recall_notice_responses", "ope
ration": "GET_RECALL_NOTICE", "data": {"recall_notice_id": "aa68813e-e4fc-4bcb-99a6-a3331da049ba"}}
18:44:45.383 [worker] PROCESSED cid=d33da180-635b-4d46-ac9c-a94397dcccdd op=GET_RECALL_NOTICE success=True store_size=6
18:44:45.385 [worker] SENT      topic=s9275_recall_notice_responses {"correlation_id": "d33da180-635b-4d46-ac9c-a94397dcccdd", "success": true, "recall_notice": {
"recall_notice_id": "aa68813e-e4fc-4bcb-99a6-a3331da049ba", "productName": "Sunrise Valley Creamy Peanut Butter, 16 oz jar", "recallingFirm": "Sunrise Valley Food
s, Inc.", "submitterEmail": "quality@sunrisevalleyfoods.com", "description": "Lots SV2451-SV2458, best-by 2027-03-11 to 2027-03-18, may contain undeclared milk fr
om a shared production line. Do not eat; return to the store for a full refund.", "category": "Undeclared Allergen", "agreedToTerms": true}}
18:44:45.421 [client] RECEIVED  topic=s9275_recall_notice_responses {"correlation_id": "d33da180-635b-4d46-ac9c-a94397dcccdd", "success": true, "recall_notice": {
"recall_notice_id": "aa68813e-e4fc-4bcb-99a6-a3331da049ba", "productName": "Sunrise Valley Creamy Peanut Butter, 16 oz jar", "recallingFirm": "Sunrise Valley Food
s, Inc.", "submitterEmail": "quality@sunrisevalleyfoods.com", "description": "Lots SV2451-SV2458, best-by 2027-03-11 to 2027-03-18, may contain undeclared milk fr
om a shared production line. Do not eat; return to the store for a full refund.", "category": "Undeclared Allergen", "agreedToTerms": true}}
18:44:45.421 [client] CHECK     correlation_id preserved: True

--- Result ---
{
  "recall_notice_id": "aa68813e-e4fc-4bcb-99a6-a3331da049ba",
  "productName": "Sunrise Valley Creamy Peanut Butter, 16 oz jar",
  "recallingFirm": "Sunrise Valley Foods, Inc.",
  "submitterEmail": "quality@sunrisevalleyfoods.com",
  "description": "Lots SV2451-SV2458, best-by 2027-03-11 to 2027-03-18, may contain undeclared milk from a shared production line. Do not eat; return to the store
 for a full refund.",
  "category": "Undeclared Allergen",
  "agreedToTerms": true
}
18:44:45.421 [client] CHECK     retrieved id matches created id: True
```

Validation and error responses (through FastAPI)

Request	Response	correlation_id
CREATE without productName	Missing field: productName	err-missing-1

CREATE with category "Spoiled"	Invalid field: category must be one of [...]	err-category-1
GET does-not-exist	Recall notice not found	err-notfound-1
DELETE_RECALL_NOTICE	Unknown operation	err-op-1

```
Xu Wang - SID4 9275
=== HW8 Part 1 error cases via FastAPI - Xu Wang (s9275) ===
--- missing field (no productName) ---
{"correlation_id":"err-missing-1","success":false,"error":"Missing field: productName"}
--- invalid category ---
{"correlation_id":"err-category-1","success":false,"error":"Invalid field: category must be one of ['Undeclared Allergen', 'Bacterial Conta
mination', 'Foreign Material', 'Mislabeling']"}
--- unknown id ---
{"correlation_id":"err-notfound-1","success":false,"error":"Recall notice not found"}
--- unknown operation ---
{"correlation_id":"err-op-1","success":false,"error":"Unknown operation"}
--- API log ---
16:12:08.525 [api]    SENT      topic=s9275_recall_notice_requests {"correlation_id": "err-missing-1", "reply_to": "s9275_recall_notice_res
ponses", "operation": "CREATE_RECALL_NOTICE", "data": {"recallingFirm": "Test Foods", "submitterEmail": "qa@testfoods.com", "description":
"Lot TF100 may contain undeclared peanuts.", "category": "Undeclared Allergen", "agreedToTerms": true}}
16:12:08.533 [api]    RECEIVED  topic=s9275_recall_notice_responses {"correlation_id": "err-missing-1", "success": false, "error": "Missing
 field: productName"}
16:12:08.533 [api]    RETURNED  cid=err-missing-1 success=False -> HTTP caller
INFO:     172.18.0.1:58050 - "POST /recall-notices HTTP/1.1" 200 OK
16:12:08.551 [api]    SENT      topic=s9275_recall_notice_requests {"correlation_id": "err-category-1", "reply_to": "s9275_recall_notice_re
sponses", "operation": "CREATE_RECALL_NOTICE", "data": {"productName": "Oat Bar", "recallingFirm": "Test Foods", "submitterEmail": "qa@test
foods.com", "description": "Lot TF100 may contain undeclared peanuts.", "category": "Spoiled", "agreedToTerms": true}}
16:12:08.556 [api]    RECEIVED  topic=s9275_recall_notice_responses {"correlation_id": "err-category-1", "success": false, "error": "Invali
d field: category must be one of ['Undeclared Allergen', 'Bacterial Contamination', 'Foreign Material', 'Mislabeling']"}
16:12:08.556 [api]    RETURNED  cid=err-category-1 success=False -> HTTP caller
INFO:     172.18.0.1:58060 - "POST /recall-notices HTTP/1.1" 200 OK
16:12:08.575 [api]    SENT      topic=s9275_recall_notice_requests {"correlation_id": "err-notfound-1", "reply_to": "s9275_recall_notice_re
sponses", "operation": "GET_RECALL_NOTICE", "data": {"recall_notice_id": "does-not-exist"}}
16:12:08.583 [api]    RECEIVED  topic=s9275_recall_notice_responses {"correlation_id": "err-notfound-1", "success": false, "error": "Recall
 notice not found"}
16:12:08.583 [api]    RETURNED  cid=err-notfound-1 success=False -> HTTP caller
INFO:     172.18.0.1:58064 - "GET /recall-notices/does-not-exist HTTP/1.1" 200 OK
16:12:08.601 [api]    SENT      topic=s9275_recall_notice_requests {"correlation_id": "err-op-1", "reply_to": "s9275_recall_notice_response
s", "operation": "DELETE_RECALL_NOTICE", "data": {}}
16:12:08.607 [api]    RECEIVED  topic=s9275_recall_notice_responses {"correlation_id": "err-op-1", "success": false, "error": "Unknown oper
ation"}
16:12:08.607 [api]    RETURNED  cid=err-op-1 success=False -> HTTP caller
INFO:     172.18.0.1:58074 - "POST /requests HTTP/1.1" 200 OK
```

Part 2 — Three-Agent Kafka Pipeline

Planner, Writer and Reviewer run as separate processes in their own terminals. No agent imports or calls another; each reads one topic, adds its field to the message, and writes the next topic. The message grows as it moves (question -> plan -> draft -> status/answer) and keeps the original correlation_id.

Sender	Reads topic	Sends to
You (send_question.py)	-	s9275_inbox (a question)
Planner (planner.py)	s9275_inbox	s9275_tasks (a 3-step plan)
Writer (writer.py)	s9275_tasks (2 partitions)	s9275_drafts (a draft answer)
Reviewer (reviewer.py)	s9275_drafts	s9275_final (status + answer)

Topic creation

docker-compose.yml — kafka-init service

```
        for t in recall_notice_requests recall_notice_responses inbox drafts final dead_letter; do
          $$T --create --if-not-exists --topic s9275_$$t --partitions 1 --replication-factor 1 --
bootstrap-server $$B
        done
        $$T --create --if-not-exists --topic s9275_tasks --partitions 2 --replication-factor 1 --
bootstrap-server $$B
        $$T --describe --bootstrap-server $$B | grep -E "^Topic"
```

```
Created topic s9275_final.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issue
s it is best to use either, but not both.
Created topic s9275_dead_letter.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issue
s it is best to use either, but not both.
Created topic s9275_tasks.
Topic: s9275_tasks      TopicId: LejsjO6xSQeqRaQKRiTssw PartitionCount: 2       ReplicationFactor: 1    Configs:
Topic: s9275_recall_notice_responses    TopicId: lEfJE3aiRNGgt-qCBnQSEw PartitionCount: 1       ReplicationFactor: 1    C
onfigs:
Topic: s9275_dead_letter        TopicId: 0Ji7gNA6QvafJPt1RcXlTQ PartitionCount: 1       ReplicationFactor: 1    Configs:
Topic: s9275_inbox      TopicId: hI-Jo-eWTwuQ6rQ58qKmJw PartitionCount: 1       ReplicationFactor: 1    Configs:
Topic: s9275_drafts     TopicId: m81BFb-qRSadMjW9g7X3dg PartitionCount: 1       ReplicationFactor: 1    Configs:
Topic: s9275_final      TopicId: fDCYZ-MxRZKUDGNV8QbFvQ PartitionCount: 1       ReplicationFactor: 1    Configs:
Topic: s9275_recall_notice_requests     TopicId: 4JyCu_orQSmLOkCVMlo10Q PartitionCount: 1       ReplicationFactor: 1    C
onfigs:
```

Agent code

planner.py — plan, forward, commit

```
            steps = parse_steps(llm(SYSTEM, message["question"], max_tokens=200))
            log(AGENT, "PLANNED", f"cid={cid} steps={steps}", cid=cid)

            out = {**message, "plan": steps, "trace": add_trace(message, AGENT)}
            send(producer, TASKS, out)
            log(AGENT, "SENT", f"cid={cid} -> {TASKS}", cid=cid)
            consumer.commit(message=msg, asynchronous=False)
```

writer.py — the Writer's work

```
def write_draft(message: dict) -> str:
    """The real work. A message without 'question' raises KeyError (the poison case)."""
    if SLOW_SECONDS:
        log(AGENT, "WORKING", f"cid={message.get('correlation_id')} "
                              f"pid={os.getpid()} sleeping {SLOW_SECONDS:.0f}s - kill window")
        time.sleep(SLOW_SECONDS)
    steps = "\n".join(f"{i}. {s}" for i, s in enumerate(message.get("plan", []), 1))
    prompt = f"Question: {message['question']}\n\nPlan:\n{steps}"
    return llm(SYSTEM, prompt, max_tokens=300)
```

reviewer.py — deterministic rules, then the model's JSON verdict

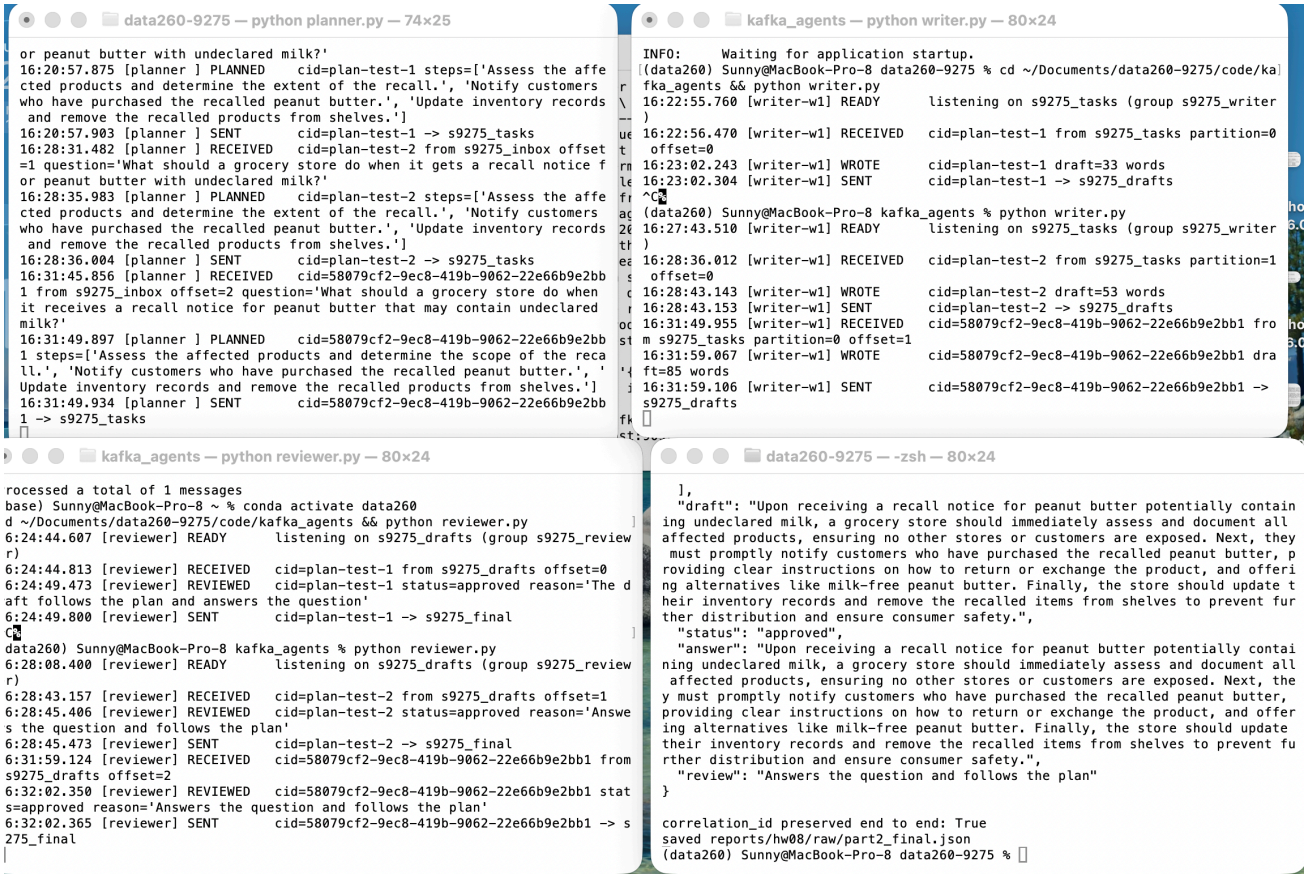
```
def rule_check(draft: str, plan: list[str]) -> str | None:
    """Return a rejection reason, or None if the draft passes the hard rules."""
    if not draft.strip():
        return "draft is empty"
    if len(draft.split()) > MAX_WORDS:
        return f"draft is longer than {MAX_WORDS} words"
    similarity = SequenceMatcher(None, _norm(draft), _norm(" ".join(plan))).ratio()
    if similarity >= COPY_THRESHOLD:
        return f"draft repeats the plan instead of answering (similarity {similarity:.2f})"
    return None


def review(message: dict) -> tuple[str, str]:
    """Return (status, reason)."""
    draft = message.get("draft", "")
    problem = rule_check(draft, message.get("plan", []))
    if problem:
        return "rejected", f"rule check: {problem}"

    steps = "\n".join(f"{i}. {s}" for i, s in enumerate(message.get("plan", []), 1))
    raw = llm(SYSTEM, f"Question: {message['question']}\n\nPlan:\n{steps}\n\nDraft:\n{draft}",
              max_tokens=120, json_mode=True)
    try:
        verdict = json.loads(raw)
        approved = verdict.get("approved") is True
        reason = str(verdict.get("reason", "")).strip() or "no reason given"
    except (ValueError, AttributeError):
        return "approved", "rule checks passed; model verdict unparseable"
```

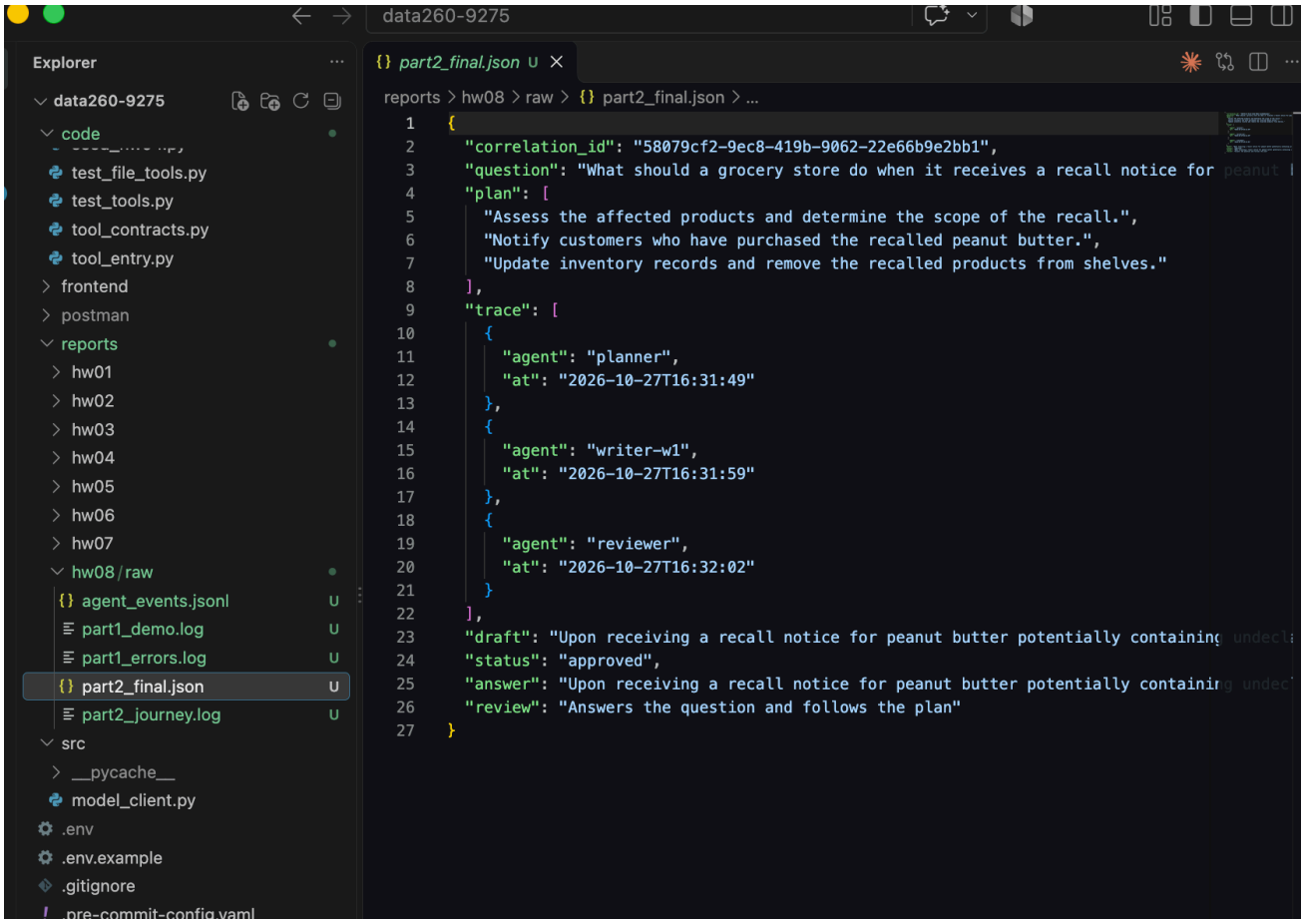
```
    return ("approved" if approved else "rejected"), reason
```

One message journey, inbox -> final



Journey — raw/part2_journey.log (all agents, ordered by time)

```
16:31:45.854 [client   ] SENT       cid=58079cf2-9ec8-419b-9062-22e66b9e2bb1 -> s9275_inbox
question='What should a grocery store do when...
16:31:45.856 [planner  ] RECEIVED   cid=58079cf2-9ec8-419b-9062-22e66b9e2bb1 from s9275_inbox
offset=2 question='What should a grocery st...
16:31:49.897 [planner  ] PLANNED    cid=58079cf2-9ec8-419b-9062-22e66b9e2bb1 steps=['Assess the
affected products and determine the scope...
16:31:49.934 [planner  ] SENT       cid=58079cf2-9ec8-419b-9062-22e66b9e2bb1 -> s9275_tasks
16:31:49.955 [writer-w1] RECEIVED   cid=58079cf2-9ec8-419b-9062-22e66b9e2bb1 from s9275_tasks
partition=0 offset=1
16:31:59.067 [writer-w1] WROTE      cid=58079cf2-9ec8-419b-9062-22e66b9e2bb1 draft=85 words
16:31:59.106 [writer-w1] SENT       cid=58079cf2-9ec8-419b-9062-22e66b9e2bb1 -> s9275_drafts
16:31:59.124 [reviewer ] RECEIVED   cid=58079cf2-9ec8-419b-9062-22e66b9e2bb1 from s9275_drafts
offset=2
16:32:02.350 [reviewer ] REVIEWED   cid=58079cf2-9ec8-419b-9062-22e66b9e2bb1 status=approved
reason='Answers the question and follows the...
16:32:02.365 [reviewer ] SENT       cid=58079cf2-9ec8-419b-9062-22e66b9e2bb1 -> s9275_final
16:32:02.379 [client   ] FINAL      cid=58079cf2-9ec8-419b-9062-22e66b9e2bb1 from s9275_final
status=approved
```



How Kafka helped the agents coordinate

Kafka is the only thing the agents share. The Planner finishes and moves on without knowing whether a Writer is running; the plan waits in s9275_tasks until one is. That decoupling showed up in testing: the first test plan sat in s9275_tasks while no Writer existed and was processed the moment I started writer.py. Each agent's consumer group remembers how far it has read, so a restarted agent continues where it stopped, and the two partitions of s9275_tasks let a second Writer share the load without any change to the Planner. The correlation_id carried in the message is what lets the client pick its own answer out of s9275_final.

Found in testing: While testing, the first Writer prompt made qwen2.5:3b copy the plan as its answer and the model Reviewer approved it. The Reviewer now rejects any draft ≥ 0.8 similar to the plan before asking the model (see AI use).

Part 3 — Failure Behavior

All fixes live in writer.py and reliability.py. Each fix has its own switch (FIX_IDEMPOTENCY, FIX_MANUAL_COMMIT, FIX_DLQ, all on by default); every "before" run turns off exactly one, so each row of the table changes one thing. Results are checked by counting drafts with the correlation_id in s9275_drafts (part3_tools.py check), not by reading log wording.

writer.py — order of operations with the fixes on

```
            # Duplicate delivery fix: an id that already produced a draft is skipped.
            if FIX_IDEMPOTENCY and store.seen(cid):
                log(AGENT, "DUPLICATE", f"cid={cid} already processed — skipped, no new draft",
                    cid=cid)
                commit(msg)
```

```
                continue

            work = (lambda m: order_test_work(m, msg)) if message.get("order_test") else
write_draft

            if FIX_DLQ:
                ok, result, attempts = run_with_retries(
                    work, message,
                    on_retry=lambda n, exc: log(AGENT, "RETRY", f"cid={cid} attempt {n}/"
                                                f"{MAX_ATTEMPTS} failed: {type(exc).__name__}:
{exc}",
                                                cid=cid))
                if not ok:
                    to_dead_letter(producer, raw, msg, AGENT, result, attempts)
                    log(AGENT, "DEAD_LTR", f"cid={cid} -> s9275_dead_letter after {attempts} "
                                           f"attempts ({result})", cid=cid)
                    commit(msg)
                    continue
                draft = result
            else:
                draft = work(message)        # before fix: an exception ends the process

            if message.get("order_test"):
                log(AGENT, "ORDERED", f"cid={cid} {draft}", cid=cid,
                    recall_notice_id=message["recall_notice_id"], seq=message["seq"])
            else:
                log(AGENT, "WROTE", f"cid={cid} draft={len(draft.split())} words", cid=cid)
                out = {**message, "draft": draft, "trace": add_trace(message, AGENT)}
                send(producer, DRAFTS, out)
                log(AGENT, "SENT", f"cid={cid} -> {DRAFTS}", cid=cid)

            store.mark(cid)      # after the output is in Kafka
            commit(msg)          # last: only now is this message done
```

I. Duplicate delivery

reliability.py — ProcessedStore: finished correlation_ids, kept in var/writer_processed.txt (survives restarts)

```
    def seen(self, cid: str) -> bool:
        return cid in self.ids

    def mark(self, cid: str) -> None:
        """Call only after the output has been sent."""
        self.ids.add(cid)
        with self.path.open("a") as fh:
            fh.write(cid + "\n")
```

Before (FIX_IDEMPOTENCY=0): the same task, same correlation_id, sent twice was processed twice — 2 model calls and 2 drafts. After: the second copy is recognised and skipped; 1 draft.

```
(data260) Sunny@MacBook-Pro-8 kafka_agents % FIX_IDEMPOTENCY=0 python writer.py
2>&1 | tee ../../reports/hw08/raw/p3_dup_before_writer.log
16:44:00.568 [writer-w1] READY      listening on s9275_tasks (group s9275_writer
) pid=53379 fixes: idempotency=False manual_commit=True dlq=True
16:44:08.146 [writer-w1] RECEIVED   cid=dup-0c594693 from s9275_tasks partition=
1 offset=1
16:44:17.357 [writer-w1] WROTE      cid=dup-0c594693 draft=73 words
16:44:17.388 [writer-w1] SENT       cid=dup-0c594693 -> s9275_drafts
16:44:17.403 [writer-w1] COMMITTED  partition=1 offset=2
16:44:17.404 [writer-w1] RECEIVED   cid=dup-0c594693 from s9275_tasks partition=
1 offset=2
16:44:26.485 [writer-w1] WROTE      cid=dup-0c594693 draft=73 words
16:44:26.516 [writer-w1] SENT       cid=dup-0c594693 -> s9275_drafts
16:44:26.523 [writer-w1] COMMITTED  partition=1 offset=3
```

kafka_agents — -zsh — 80×24

```
ther distribution and ensure consumer safety.",
  "status": "approved",
  "answer": "Upon receiving a recall notice for peanut butter potentially contai
ning undeclared milk, a grocery store should immediately assess and document all
 affected products, ensuring no other stores or customers are exposed. Next, the
y must promptly notify customers who have purchased the recalled peanut butter,
providing clear instructions on how to return or exchange the product, and offer
ing alternatives like milk-free peanut butter. Finally, the store should update
their inventory records and remove the recalled items from shelves to prevent fu
rther distribution and ensure consumer safety.",
  "review": "Answers the question and follows the plan"
}

correlation_id preserved end to end: True
saved reports/hw08/raw/part2_final.json
(data260) Sunny@MacBook-Pro-8 data260-9275 % cd ~/Documents/data260-9275/code/ka
fka_agents
python part3_tools.py replay
=== HW8 Part 3 - Xu Wang (SID4 9275, PREFIX s9275) ===
16:44:08.142 [part3   ] REPLAY     send #1 cid=dup-0c594693 -> s9275_tasks
16:44:08.150 [part3   ] REPLAY     send #2 cid=dup-0c594693 -> s9275_tasks

wait until the Writer is idle, then:  python part3_tools.py check dup-0c594693
(data260) Sunny@MacBook-Pro-8 kafka_agents %
```

```
(data260) Sunny@MacBook-Pro-8 kafka_agents % python writer.py 2>&1 | tee ../../reports/hw08/raw/p3_dup_after_writer.log
16:45:37.404 [writer-w1] READY      listening on s9275_tasks (group s9275_writer) pid=54688 fixes: idempotency=True manual_commit=True dlq=True
16:45:55.608 [writer-w1] RECEIVED   cid=dup-9bd1743b from s9275_tasks partition=1 offset=3
16:46:04.141 [writer-w1] WROTE      cid=dup-9bd1743b draft=73 words
16:46:04.159 [writer-w1] SENT       cid=dup-9bd1743b -> s9275_drafts
16:46:04.169 [writer-w1] COMMITTED  partition=1 offset=4
16:46:04.169 [writer-w1] RECEIVED   cid=dup-9bd1743b from s9275_tasks partition=1 offset=4
16:46:04.170 [writer-w1] DUPLICATE  cid=dup-9bd1743b already processed - skipped, no new draft
16:46:04.171 [writer-w1] COMMITTED  partition=1 offset=5
```

```
(data260) Sunny@MacBook-Pro-8 kafka_agents % python part3_tools.py replay
=== HW8 Part 3 - Xu Wang (SID4 9275, PREFIX s9275) ===
16:45:55.602 [part3   ] REPLAY     send #1 cid=dup-9bd1743b -> s9275_tasks
16:45:55.612 [part3   ] REPLAY     send #2 cid=dup-9bd1743b -> s9275_tasks

wait until the Writer is idle, then:  python part3_tools.py check dup-9bd1743b
(data260) Sunny@MacBook-Pro-8 kafka_agents %
```

II. Consumer death

common.py — make_consumer(): manual commit; auto-commit only for the before run

```
    config = {
        "bootstrap.servers": BOOTSTRAP,
        "group.id": group_id,
        "auto.offset.reset": offset_reset,
        "enable.auto.commit": auto_commit,
        # A killed consumer is noticed after 10 s instead of the default 45 s,
        # so its partitions move to the restarted consumer quickly.
        "session.timeout.ms": 10000,
    }
    if auto_commit:
        config["auto.commit.interval.ms"] = 1000
```

The Writer was started with SLOW_SECONDS=20, killed with kill -9 while working on the task, and restarted. Before (auto-commit every second): the offset was committed before the work finished (LAG 0), the restarted Writer received nothing and the task was lost — 0 drafts. After: the offset was still uncommitted (LAG 1); the restarted Writer re-read the same offset 5, finished it and only then committed offset 6 — 1 draft.

```
(data260) Sunny@MacBook-Pro-8 kafka_agents % FIX_MANUAL_COMMIT=0 SLOW_SECONDS=20 python writer.py 2>&1 | tee ../../reports/hw08/raw/p3_kill_before_writer.log
17:26:37.224 [writer-w1] READY      listening on s9275_tasks (group s9275_writer) pid=80376 fixes: idempotency=True manual_commit=False dlq=True
17:26:48.759 [writer-w1] RECEIVED   cid=kill-a0c7ae0b from s9275_tasks partition=0 offset=4
17:26:48.760 [writer-w1] WORKING    cid=kill-a0c7ae0b pid=80376 sleeping 20s - kill window
zsh: killed     FIX_MANUAL_COMMIT=0 SLOW_SECONDS=20 python writer.py 2>&1 |
zsh: done       tee ../../reports/hw08/raw/p3_kill_before_writer.log
(data260) Sunny@MacBook-Pro-8 kafka_agents %
```

```
(data260) Sunny@MacBook-Pro-8 kafka_agents % SLOW_SECONDS=20 python writer.py 2>&1 | tee ../../reports/hw08/raw/p3_kill_after_writer.log
17:29:19.339 [writer-w1] READY      listening on s9275_tasks (group s9275_writer) pid=82546 fixes: idempotency=True manual_commit=True dlq=True
17:29:30.738 [writer-w1] RECEIVED   cid=kill-e2ca57a0 from s9275_tasks partition=0 offset=5
17:29:30.743 [writer-w1] WORKING    cid=kill-e2ca57a0 pid=82546 sleeping 20s - kill window
zsh: killed     SLOW_SECONDS=20 python writer.py 2>&1 |
zsh: done       tee ../../reports/hw08/raw/p3_kill_after_writer.log
(data260) Sunny@MacBook-Pro-8 kafka_agents %
(data260) Sunny@MacBook-Pro-8 data260-9275 % echo "Xu Wang - SID4 9275"
tail -n 1 ~/Documents/data260-9275/reports/hw08/raw/p3_kill_before_offsets.log
tail -n 1 ~/Documents/data260-9275/reports/hw08/raw/p3_kill_after_offsets.log
Xu Wang - SID4 9275
17:33:22.084 [part3   ] CHECK      cid=kill-a0c7ae0b drafts in s9275_drafts: 0
17:33:27.725 [part3   ] CHECK      cid=kill-e2ca57a0 drafts in s9275_drafts: 1
(data260) Sunny@MacBook-Pro-8 data260-9275 %
```

III. Poison message

reliability.py — bounded retries (the dead-letter send is in the writer.py block above)

```
def run_with_retries(fn: Callable[[Dict[str, Any]], Any], message: Dict[str, Any],
                     on_retry: Callable[[int, Exception], None]) -> Tuple[bool, Any, int]:
    """Return (ok, result_or_error_text, attempts_used)."""
    last_error: Exception | None = None
    for attempt in range(1, MAX_ATTEMPTS + 1):
        try:
            return True, fn(message), attempt
        except Exception as exc:  # noqa: BLE001 - any failure counts as a failed attempt
            last_error = exc
            on_retry(attempt, exc)
            if attempt < MAX_ATTEMPTS:
                time.sleep(BACKOFF_S * attempt)
    return False, f"{type(last_error).__name__}: {last_error}", MAX_ATTEMPTS
```

A task without a question field was sent, with a normal task behind it in the same partition. Before (FIX_DLQ=0): KeyError killed the Writer; the restart re-read offset 7 and died again; partition 1 stayed at LAG 2 and the normal task was blocked. After: 3 attempts, then the original message and the error went to s9275_dead_letter, offset 8 was committed and the normal task was processed (1 draft).

s9275_dead_letter record (original message omitted here)

```
{
  "correlation_id": "poison-73ba33df",
  "failed_agent": "writer-w1",
  "source_topic": "s9275_tasks",
  "source_partition": 1,
  "source_offset": 7,
  "attempts": 3,
  "error": "KeyError: 'question'"
}
```

```
(data260) Sunny@MacBook-Pro-8 kafka_agents % cd ~/Documents/data260-9275/code/ka
fka_agents
FIX_DLQ=0 python writer.py 2>&1 | tee ../../reports/hw08/raw/p3_poison_before_wr
iter.log
17:37:08.372 [writer-w1] READY      listening on s9275_tasks (group s9275_writer
) pid=88670 fixes: idempotency=True manual_commit=True dlq=False
17:37:17.482 [writer-w1] RECEIVED   cid=poison-73ba33df from s9275_tasks partiti
on=1 offset=7
Traceback (most recent call last):
  File "/Users/Sunny/Documents/data260-9275/code/kafka_agents/writer.py", line 1
56, in <module>
    main()
  File "/Users/Sunny/Documents/data260-9275/code/kafka_agents/writer.py", line 1
36, in main
    draft = work(message)        # before fix: an exception ends the process
            ^^^^^^^^^^^^
  File "/Users/Sunny/Documents/data260-9275/code/kafka_agents/writer.py", line 6
6, in write_draft
    prompt = f"Question: {message['question']}\n\nPlan:\n{steps}"
                         ~~~~~~~^^^^^^^^^^^^
KeyError: 'question'
(data260) Sunny@MacBook-Pro-8 kafka_agents % FIX_DLQ=0 python writer.py 2>&1 | t
ee -a ../../reports/hw08/raw/p3_poison_before_writer.log
17:37:40.209 [writer-w1] READY      listening on s9275_tasks (group s9275_writer
) pid=89175 fixes: idempotency=True manual_commit=True dlq=False
17:37:40.277 [writer-w1] RECEIVED   cid=poison-73ba33df from s9275_tasks partiti
on=1 offset=7
Traceback (most recent call last):
  File "/Users/Sunny/Documents/data260-9275/code/kafka_agents/writer.py", line 1
56, in <module>
    main()
  File "/Users/Sunny/Documents/data260-9275/code/kafka_agents/writer.py", line 1
36, in main
    draft = work(message)        # before fix: an exception ends the process
            ^^^^^^^^^^^^
  File "/Users/Sunny/Documents/data260-9275/code/kafka_agents/writer.py", line 6
6, in write_draft
    prompt = f"Question: {message['question']}\n\nPlan:\n{steps}"
                         ~~~~~~~^^^^^^^^^^^^
KeyError: 'question'
(data260) Sunny@MacBook-Pro-8 kafka_agents %
```

```
(data260) Sunny@MacBook-Pro-8 kafka_agents % python writer.py 2>&1 | tee ../../r
eports/hw08/raw/p3_poison_after_writer.log
17:39:07.778 [writer-w1] READY      listening on s9275_tasks (group s9275_writer
) pid=90295 fixes: idempotency=True manual_commit=True dlq=True
17:39:07.860 [writer-w1] RECEIVED   cid=poison-73ba33df from s9275_tasks partiti
on=1 offset=7
17:39:07.860 [writer-w1] RETRY      cid=poison-73ba33df attempt 1/3 failed: KeyE
rror: 'question'
17:39:08.363 [writer-w1] RETRY      cid=poison-73ba33df attempt 2/3 failed: KeyE
rror: 'question'
17:39:09.369 [writer-w1] RETRY      cid=poison-73ba33df attempt 3/3 failed: KeyE
rror: 'question'
17:39:09.390 [writer-w1] DEAD_LTR   cid=poison-73ba33df -> s9275_dead_letter aft
er 3 attempts (KeyError: 'question')
17:39:09.393 [writer-w1] COMMITTED  partition=1 offset=8
17:39:09.394 [writer-w1] RECEIVED   cid=after-poison-542fb13e from s9275_tasks p
artition=1 offset=8
17:39:22.366 [writer-w1] WROTE      cid=after-poison-542fb13e draft=73 words
17:39:22.395 [writer-w1] SENT       cid=after-poison-542fb13e -> s9275_drafts
17:39:22.409 [writer-w1] COMMITTED  partition=1 offset=9
```

IV. Ordering — two partitions, two Writers

part3_tools.py — the only difference between the two runs: the message key

```
        if run == "keyed":
            # Fix: key by the related entity -> one partition per recall -> order kept.
            producer.produce(TASKS, key=m["recall_notice_id"], value=json.dumps(message))
        else:
            # Before: no key; spread round-robin over the 2 partitions.
            producer.produce(TASKS, value=json.dumps(message), partition=i % 2)
```

Using VERIFY_SEED 269275, 20 related tasks were generated: 4 recalls × 5 steps (notice received, lots verified, shelves cleared, customers notified, recall closed), interleaved but always sent in step order. Two Writers (w1, w2) in group s9275_writer each owned one partition of s9275_tasks. In these runs the Writer replaces the model call with 0.1–0.6 s of fixed work per (recall, step), so the key is the only variable. The no-key run spreads tasks round-robin over the two partitions (librdkafka's sticky partitioner would otherwise put a fast burst of keyless messages in one partition and hide the problem).

Run	Recall	Partitions	Writers	Step order processed	Violations
no key	RN-c6810e	0,1	w1,w2	[1, 2, 4, 3, 5]	1
no key	RN-c973f5	0,1	w1,w2	[1, 3, 2, 5, 4]	2
no key	RN-c97b16	0,1	w1,w2	[1, 2, 3, 4, 5]	0
no key	RN-efb54f	0,1	w1,w2	[1, 2, 3, 4, 5]	0
key = recall_notice_id	RN-c6810e	0	w2	[1, 2, 3, 4, 5]	0
key = recall_notice_id	RN-c973f5	1	w1	[1, 2, 3, 4, 5]	0
key = recall_notice_id	RN-c97b16	1	w1	[1, 2, 3, 4, 5]	0
key = recall_notice_id	RN-efb54f	0	w2	[1, 2, 3, 4, 5]	0

Total violations: 3 without a key, 0 with the key. A violation is a step processed after a higher-numbered step of the same recall.

```
(data260) Sunny@MacBook-Pro-8 kafka_agents % docker exec s9275-kafka /opt/kafka/
bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group s9275_writer | tee ../../reports/hw08/raw/p3_order_assignme
nt.log

GROUP           TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
          CONSUMER-ID                                     HOST            CLIENT-I
D
s9275_writer    s9275_tasks     0          6               6               0
          rdkafka-bce9de13-b25b-42b0-b03a-32703e07c3d7 /192.168.65.1   rdkafka
s9275_writer    s9275_tasks     1          9               9               0
          rdkafka-dde27ba4-eb0d-417f-88ba-5e92749db3d2 /192.168.65.1   rdkafka%
(data260) Sunny@MacBook-Pro-8 kafka_agents %
```

```
(data260) Sunny@MacBook-Pro-8 kafka_agents % python part3_tools.py analyze nokey
 | tee ../../reports/hw08/raw/p3_order_nokey.log
=== HW8 Part 3 - Xu Wang (SID4 9275, PREFIX s9275) ===
run=nokey: 20 of 20 processed
recall     partitions  writers  seq in processing order    violations
RN-c6810e  0,1         w1,w2    [1, 2, 4, 3, 5]            1
RN-c973f5  0,1         w1,w2    [1, 3, 2, 5, 4]            2
RN-c97b16  0,1         w1,w2    [1, 2, 3, 4, 5]            0
RN-efb54f  0,1         w1,w2    [1, 2, 3, 4, 5]            0
total order violations: 3
```

```
(data260) Sunny@MacBook-Pro-8 kafka_agents % python part3_tools.py analyze keyed
 | tee ../../reports/hw08/raw/p3_order_keyed.log
=== HW8 Part 3 - Xu Wang (SID4 9275, PREFIX s9275) ===
run=keyed: 20 of 20 processed
recall     partitions  writers  seq in processing order    violations
RN-c6810e  0           w2       [1, 2, 3, 4, 5]            0
RN-c973f5  1           w1       [1, 2, 3, 4, 5]            0
RN-c97b16  1           w1       [1, 2, 3, 4, 5]            0
RN-efb54f  0           w2       [1, 2, 3, 4, 5]            0
total order violations: 0
```

Failure table

Failure scenario	Before fix	After fix
Duplicate delivery	Same task + correlation_id sent twice -> processed twice, 2 drafts (dup-0c594693)	Second copy -> DUPLICATE skipped, 1 draft (dup-9bd1743b)
Consumer killed mid-work	Auto-commit: offset committed before work finished (LAG 0); restart gets nothing; 0 drafts — lost (kill-a0c7ae0b)	Commit after send: LAG 1 after crash; restart re-reads offset 5, commits 6; 1 draft (kill-e2ca57a0)
Poison message	KeyError kills Writer; restart dies on the same offset 7; LAG 2 — next task blocked	3 attempts -> s9275_dead_letter -> commit; next task processed, 1 draft
Two partitions / two consumers	No key: every recall split over partitions 0,1 and w1,w2; 3 order violations	Key = recall_notice_id: one partition and one Writer per recall; steps 1-5 in order; 0 violations

Screenshots for every row are above; full logs in reports/hw08/RUN_LOG.txt and raw/.

V. Comparison with the HW2 in-process graph

In HW2 the LangGraph supervisor called each node as a function in one Python process; a step started only when the previous call returned.

Benefits gained by moving to Kafka:

- Decoupled failure. When a Writer died, the Planner and Reviewer kept running and the unfinished task stayed in s9275_tasks; the restarted Writer finished it (Part 3 II). In HW2 an exception in one node ended the whole run.
- Independent scaling. A second Writer joined group s9275_writer and took one of the two partitions with no change to the Planner or Reviewer (Part 3 IV). In HW2 every step ran at the speed of one process.

New responsibilities I now handle myself:

- Duplicates and offsets. Kafka delivers at least once, so the same task can arrive twice; I had to add an idempotency store and decide exactly when to commit the offset (after the output is sent). A function call runs once and has no offset.
- Ordering and bad input. Order is only guaranteed inside one partition, so related messages need a key; and a message that always fails must be retried a bounded number of times and moved to a dead-letter topic, or it blocks its partition. In HW2 steps were ordered by the call stack and an error simply raised.

Verification

make verify-hw08 runs verify_hw08.py on commit cdf9602 and writes reports/hw08/verification.json: 30 passed, 0 failed. Live checks: containers running, 7 topics, s9275_tasks has 2 partitions, CREATE -> GET round trip through Kafka and through FastAPI, three error cases, correlation_id echoed every time. Unit and evidence checks: Reviewer copy rule, idempotency store, 3 retry attempts, and the before/after result of all four failure rows. The script uses a temporary directory and does not change application code.

```
30 passed, 0 failed -> reports/hw08/verification.json
```

AI Use

1. What I used an AI assistant for, and what I did myself.

I used Claude (Anthropic) as a coding assistant.

I summarized the assignment into concrete tasks, proposed the design (one compose stack for Kafka + worker + FastAPI, a growing JSON message for the three agents, one switch per Part 3 fix so each "before" run changes only one thing).

It wrote the framework of code one file at a time: the RecallNotice worker and FastAPI wrapper, homework_demo.py, the Planner, Writer and Reviewer, the reliability helpers, the Part 3 test tool.

I filled up the core parts of each file.

I ran every command on my Mac (Docker, the four agent terminals, every before/after failure run), chose to run the consumer-death test by hand, step by step, instead of an automated script, took every screenshot, and checked each result against the expected output before committing.

2. One AI-produced output that was wrong.

The first Writer prompt asked the model to answer "covering each step of the plan in order". qwen2.5:3b answered by copying the three plan lines word for word, and the Reviewer (also the model) approved it, so the first end-to-end run delivered "status": "approved" with an answer that answered nothing.

3. How I detected it.

I read the final JSON from s9275_final for the first test message (plan-test-1): the draft and answer fields were identical to the plan list, and the review reason was "The draft follows the plan and answers the question". The pipeline "worked"; only reading the content showed the failure.

4. What I changed and why it works now.

Two changes. The Writer prompt now asks for one paragraph of 3–5 full sentences, uses the plan only as an outline and forbids copying the plan lines or a numbered list. The Reviewer got a deterministic rule that runs before the model: if the draft's normalized text is ≥ 0.8 similar to the plan (difflib), it is rejected. The copied draft scores 1.00 and is rejected; the next runs produced 53- and 85-word answers that pass the rule and were approved. verify_hw08.py checks both cases, so the rule cannot silently disappear.

A second issue I verified independently: after the compose file changed, Docker recreated the Kafka container and every topic was gone, which made homework_demo.py fail with "no partition assignment". Listing the topics confirmed it; the fix is the kafka-init service, which recreates all seven topics before the worker and API start.